Probability Mass Functions (PMF) and Discrete Probability

Table of Contents

Introduction

Probability is the mathematics of uncertainty. When we don't know exactly what will happen, we can use probability to describe how likely different outcomes are.

Real-world example: Imagine you're searching for a lost submarine in the ocean. You don't know exactly where it is, but some locations are more likely than others based on currents, last known position, etc.

Random Variables

What is a Random Variable?

A **random variable** is a variable whose value is uncertain until we observe it. We use capital letters like X, Y, Z to denote random variables.

Types of Random Variables:

- **Discrete**: Can only take specific, countable values (like rolling a die: 1, 2, 3, 4, 5, 6)
- **Continuous**: Can take any value in a range (like temperature: 20.1°C, 20.15°C, etc.)

Example: Submarine Location

If we divide the ocean into a 16×16 grid, the submarine's location X can be any of 256 grid squares. This is a discrete random variable because there are countable possibilities.

```
Grid square X = [x, y] where x and y are integers from 0 to 15
Sample space: {[0,0], [0,1], ..., [15,15]} = 256 possibilities
```

Probability Mass Function (PMF)

Definition

A **Probability Mass Function (PMF)** tells us the probability that a discrete random variable equals each specific value.

Mathematical notation: f_X(x) = P(X = x)

This reads as: "The PMF of X evaluated at x equals the probability that X equals x"

Visual Understanding

Think of a PMF as a map showing how "probability mass" is distributed across all possible outcomes. Higher values mean that outcome is more likely.

```
Example: Rolling a fair 6-sided die
P(X = 1) = 1/6 ≈ 0.167
P(X = 2) = 1/6 ≈ 0.167
P(X = 3) = 1/6 ≈ 0.167
P(X = 4) = 1/6 ≈ 0.167
P(X = 5) = 1/6 ≈ 0.167
P(X = 6) = 1/6 ≈ 0.167
```

Properties of Valid PMFs

For a function to be a valid PMF, it must satisfy TWO properties:

Property 1: All probabilities are between 0 and 1

```
0 ≤ P(X = x) ≤ 1  for all x
```

Why? Probability of 0 means impossible, 1 means certain. Nothing can be more certain than certain or less possible than impossible!

Property 2: All probabilities sum to 1

```
Σ P(X = x) = 1  (sum over all possible x)
```

Why? Something must happen! The probabilities of all possible outcomes must add up to 100% (or 1.0).

Example: Checking if a PMF is valid

Valid PMF:

```
pmf = np.array([[0.1, 0.2],
                [0.3, 0.4]])
# All values between 0 and 1: ✓
# Sum = 0.1 + 0.2 + 0.3 + 0.4 = 1.0: ✓
# This is VALID
```

Invalid PMF:

```
pmf = np.array([[0.1, 0.2],
                [0.3, 0.5]])
# All values between 0 and 1: ✓
# Sum = 0.1 + 0.2 + 0.3 + 0.5 = 1.1: ✗
# This is INVALID (probabilities sum to more than 1)
```

Computing Probabilities

Single Outcome

Question: What's the probability the submarine is at grid square [3, 4]?

Answer: Just look up that position in the PMF!

```
probability = submarine_pmf[3, 4]
```

Multiple Outcomes (Events)

An **event** is a set of outcomes. To find the probability of an event, sum the probabilities of all outcomes in that event.

Rule: P(A or B) = P(A) + P(B) when A and B are mutually exclusive (can't both happen)

Example 1: Probability submarine is in row 5

```
# Sum all probabilities in row 5
prob = np.sum(submarine_pmf[5, :])
```

Example 2: Probability submarine has x ≥ 4

```
# Sum all probabilities where x coordinate is 4 or greater
prob = np.sum(submarine_pmf[4:, :])
```

Example 3: Probability submarine is in a box [2,2] to [4,5]

```
# Sum all probabilities in the rectangular region (inclusive!)
prob = np.sum(submarine_pmf[2:5, 2:6])  # Remember: Python slicing is [start:e
```

Complementary Probability

Rule: P(NOT A) = 1 - P(A)

Example: Probability submarine is NOT at [2,4]

```
prob_not = 1 - submarine_pmf[2, 4]
```

Odds and Log-Odds

Odds

Odds are another way to express probability:

```
Odds(A) = P(A) / P(NOT A) = P(A) / (1 - P(A))
```

Interpretation:

- Odds = 1: Event is equally likely to happen or not happen (50/50)
- Odds = 3: Event is 3 times more likely to happen than not happen
- Odds = 0.5: Event is half as likely to happen compared to not happening

Example:

```
p = 0.75  # 75% probability
odds = p / (1 - p) = 0.75 / 0.25 = 3
# "3 to 1 odds" or "3:1"
```

Log-Odds (Logits)

Log-odds (or logits) is the natural logarithm of the odds:

```
Logit(A) = ln(Odds(A)) = ln(P(A) / (1 - P(A)))
```

Why use log-odds?

- Converts probabilities from [0,1] to (-∞, +∞)
- Makes calculations easier in statistics and machine learning
- Symmetric around 0

Example:

```
import numpy as np

p = 0.5
logit = np.log(p / (1 - p)) = np.log(1) = 0  # 50/50 → 0

p = 0.75
logit = np.log(0.75 / 0.25) = np.log(3) ≈ 1.099

p = 0.25
logit = np.log(0.25 / 0.75) = np.log(0.333) ≈ -1.099
```

Change in Log-Odds

When comparing two hypotheses, the **change** in log-odds tells us how much more (or less) likely one is compared to the other:

```
Δ logit = logit(B) - logit(A)
```

Example: Comparing two search regions

```
p_box1 = 0.3  # Probability in box 1
p_box2 = 0.6  # Probability in box 2

logit1 = np.log(p_box1 / (1 - p_box1))
logit2 = np.log(p_box2 / (1 - p_box2))
delta_logit = logit2 - logit1  # Positive means box 2 more likely
```

Expected Value

The **expected value** (or mean) is the average outcome if we repeated the random process many times.

Definition

```
E[X] = Σ x · P(X = x)  (sum over all possible x)
```

In words: Multiply each possible value by its probability, then sum everything up.

Example: Expected Value of a Die Roll

```
E[X] = 1·(1/6) + 2·(1/6) + 3·(1/6) + 4·(1/6) + 5·(1/6) + 6·(1/6)
     = (1 + 2 + 3 + 4 + 5 + 6) / 6
     = 21 / 6
     = 3.5
```

Note: The expected value doesn't have to be a possible outcome! You can't roll 3.5, but on average, you get 3.5.

Expected Value of a Function

Important Rule: E[f(X)] ≠ f(E[X]) in general!

```
E[f(X)] = Σ f(x) · P(X = x)
```

Example: Expected value of distance

```
# WRONG approach:
expected_location = compute_expected_location(pmf)
expected_distance = compute_distance(expected_location)  # This is f(E[X])

# CORRECT approach:
expected_distance = 0
for all grid squares (x, y):
    distance = compute_distance(x, y)
    expected_distance += distance * pmf[x, y]  # This is E[f(X)]
```

Practical Examples with NumPy

Example 1: Create a Simple PMF

```
import numpy as np

# Create a 4x4 grid PMF
# Higher probability near center
pmf = np.array([
    [0.02, 0.03, 0.03, 0.02],
    [0.03, 0.10, 0.10, 0.03],
    [0.03, 0.10, 0.10, 0.03],
    [0.02, 0.03, 0.03, 0.02]
])

# Verify it's valid
print("Sum:", np.sum(pmf))  # Should be 1.0
print("Min:", np.min(pmf))  # Should be ≥ 0
print("Max:", np.max(pmf))  # Should be ≤ 1
```

Example 2: Compute Probabilities

```
# Probability at specific location
p_at_2_2 = pmf[2, 2]
print(f"P(X=[2,2]) = {p_at_2_2}")

# Probability in top row
p_top_row = np.sum(pmf[0, :])
print(f"P(x=0) = {p_top_row}")

# Probability x is even (0 or 2)
p_even = np.sum(pmf[[0, 2], :])
print(f"P(x is even) = {p_even}")

# Probability in top-left 2x2 box
p_box = np.sum(pmf[0:2, 0:2])
print(f"P(box [0,0] to [1,1]) = {p_box}")
```

Example 3: Expected Location

```
# Create coordinate grids
x_coords = np.arange(4)
y_coords = np.arange(4)
xx, yy = np.meshgrid(x_coords, y_coords)

# Expected x coordinate
expected_x = np.sum(xx * pmf)

# Expected y coordinate
expected_y = np.sum(yy * pmf)

expected_location = np.array([expected_x, expected_y])
print(f"Expected location: {expected_location}")
```

Example 4: Expected Distance from a Point

```
# Calculate expected Euclidean distance from point [1, 1]
station = np.array([1, 1])

# Compute distance at each grid point
distances = np.sqrt((xx - station[0])**2 + (yy - station[1])**2)

# Expected distance
expected_distance = np.sum(distances * pmf)
print(f"Expected distance: {expected_distance}")
```

Example 5: Conditional Selection

```
# Probability that y is divisible by 3
# y can be 0, 1, 2, 3
# Divisible by 3: y = 0 or y = 3

mask = (yy % 3 == 0)  # Boolean mask: True where y divisible by 3
p_div_3 = np.sum(pmf[mask])
print(f"P(y divisible by 3) = {p_div_3}")

# Alternative using slicing
p_div_3_alt = np.sum(pmf[:, [0, 3]])
print(f"Alternative: {p_div_3_alt}")
```

Example 6: Complex Event

```
# Probability: (x < 2 AND y < 2) OR (x ≥ 2 AND y ≥ 2)
mask1 = (xx < 2) & (yy < 2)  # Top-left quadrant
mask2 = (xx >= 2) & (yy >= 2)  # Bottom-right quadrant
combined_mask = mask1 | mask2  # OR

probability = np.sum(pmf[combined_mask])
print(f"P(diagonal quadrants) = {probability}")
```

Common Mistakes to Avoid

Mistake 1: Confusing indices with coordinates

```
# If working with [x, y] coordinates:
# x is the column, y is the row
# But NumPy arrays are indexed as [row, column]!

# WRONG:
probability = pmf[x, y]  # If x, y are x-y coordinates

# CORRECT:
probability = pmf[y, x]  # Swap them!
# OR use consistent [x, y] = [row, col] convention
```

Mistake 2: Off-by-one errors in slicing

```
# "Box from [2,2] to [4,5] inclusive"

# WRONG:
box = pmf[2:4, 2:5]  # Stops at 3, not 4!

# CORRECT:
box = pmf[2:5, 2:6]  # Include endpoints: [start:end+1]
```

Mistake 3: Computing E[f(X)] incorrectly

```
# Want: Expected value of X²

# WRONG:
expected_x = np.sum(xx * pmf)
expected_x_squared = expected_x ** 2  # This is (E[X])², not E[X²]!

# CORRECT:
expected_x_squared = np.sum((xx ** 2) * pmf)  # This is E[X²]
```

Summary

Key Concepts:

1. **PMF**: Maps each possible value to its probability
2. **Valid PMF**: All probabilities in [0,1] and sum to 1
3. **Events**: Sum probabilities of outcomes in the event
4. **Expected Value**: Weighted average using probabilities
5. **E[f(X)] ≠ f(E[X])**: Apply function first, then compute expectation

Key NumPy Operations:

- `np.sum(pmf)` : Sum all probabilities
- `np.sum(pmf[mask])` : Sum probabilities matching a condition
- `np.sum(pmf[start:end, :])` : Sum over rows
- `np.sum(values * pmf)` : Compute expected value

Practice Tips:

1. Always verify your PMF sums to 1.0
2. Visualize the problem with small examples first
3. Use boolean masks for complex conditions
4. Double-check index ordering (row vs. column, x vs. y)
5. Test your code with simple cases where you know the answer

---

Further Reading

- Probability axioms and set theory
- Marginal and joint distributions
- Independence of random variables
- Variance and standard deviation
- Common discrete distributions (Binomial, Poisson, Geometric)